Youzhi Zhang

Presenter · slides 14–21 · about 8:10 of narration

> THE PROJECT STORY
>
> We wanted to study dispatch decisions. April's data described aggregate patterns rather than individual journeys, so we built plausible evenings from evidence and explicit assumptions. We checked the implementation, replayed identical evenings under different transfer rules, and found a conditional trade-off between timely service and Lyft use. Our next work tests how robust those findings are.

Contents

1 Your section 1

2 Essential preparation reading 2

3 Concepts you must explain confidently 2

4 Questions you lead on 2

5 Your narration (exact words) 4

6 Manual verification activity: One decision moment on a small designed evening 8

7 Rehearsal and readiness checklist 13

1 Your section

Slides 14–21: Three components have different jobs → We checked implementation before interpreting outcomes.

Purpose: Architecture, decision cycle, mathematics, transfer rules, worked example, postponement issue and verification.

You start immediately after the handoff sentence: "Those inputs define the constructed evening. Youzhi will explain how the model makes decisions as that evening unfolds." Begin with slide 14's first sentence; do not re-introduce yourself.

Your handoff sentence (end of slide 21): "We then replayed the same evenings under different transfer rules. Vignesh will explain what changed, what it means and what remains uncertain."

2 Essential preparation reading

Read these in the shared guide (`01_Team_Presentation_Guide.pdf`, one folder up) and the methodology PDF:

- Slide 14 · Three components have different jobs: guide, slide 14 (guide p. 29); methodology §1.2 Three things kept apart, p. 6; §Fig. 1 The system in one picture, p. 7; §4 The simulator contract in plain language, p. 19
- Slide 15 · The same decision cycle repeats as the evening unfolds: guide, slide 15 (guide p. 30); methodology §4.3 Same-second events, p. 21; §4.4 Epochs: when the dispatcher re-plans, p. 21; §4.5 The Lyft rule pass, p. 21
- Slide 16 · The optimizer compares the consequences of candidate plans: guide, slide 16 (guide p. 32); methodology §5.2 What a "good" plan means, p. 26; §5.3 The route-selection model, p. 28; §5.4 Worked example, p. 29; §5.5 What "optimal" means here, p. 31
- Slide 17 · We changed the transfer decision, not the van dispatcher: guide, slide 17 (guide p. 34); methodology §4.5 The Lyft rule pass, p. 21; §6.2 The transfer path, p. 34
- Slide 18 · What does 'Lyft arrives sooner' actually compare?: guide, slide 18 (guide p. 36); methodology §4.6 Transfers and the Lyft service, p. 22; §6.2 The transfer path, p. 34
- Slide 19 · A cheap postponement price could keep a rider waiting: guide, slide 19 (guide p. 37); methodology §7.4 A property of the objective that M1 exposed, p. 37; §7.5 The pending-price decision experiment, p. 37
- Slide 20 · Replay the same night; change one rule: guide, slide 20 (guide p. 39); methodology §3.10 Patterns, seeds and pairing, p. 19; §7.7 M3: the L5 pilot and the 30-night batch, p. 40
- Slide 21 · We checked implementation before interpreting outcomes: guide, slide 21 (guide p. 40); methodology §7.2 What was verified, p. 36; §7.7 M3 verification, p. 40
- Everyone: guide sections "What every member must be able to explain" and "Quick Q&A navigation".

3 Concepts you must explain confidently

- the three components and what the dispatcher can and cannot see
- the eight-step decision cycle; one rule pass and at most one re-plan
- the objective blocks and what 'optimal' means (best among generated candidates at that moment)
- the six rules' exact definitions and eligibility
- the worked transfer; the postponement-price flaw and its partial fix
- paired replay; verification vs validation, and that AI wrote the code and checks

4 Questions you lead on

Answer these first; route anything else to its owner (guide, Quick Q&A navigation).

- Simulator, optimizer and rule: who does what — guide slides guide, slide 14 (guide p. 29), guide, slide 15 (guide p. 30); backups 32
- The objective, 'optimal', candidate restriction — guide slides guide, slide 16 (guide p. 32); backups 31, 37

- Exact rule definitions and eligibility — guide slides guide, slide 17 (guide p. 34); backups 30
- The worked transfer timeline — guide slides guide, slide 18 (guide p. 36)
- The postponement-price flaw and fix — guide slides guide, slide 19 (guide p. 37); backups 34
- Paired replay; 30 nights, not 180 — guide slides guide, slide 20 (guide p. 39); backups 36
- Verification vs validation; AI-written code — guide slides guide, slide 21 (guide p. 40); backups 37

5 Your narration (exact words)

Slide 14 · Three components have different jobs

about 65 s · open slide

SAY (exact narration)

Three components have different jobs. The simulator advances time and reveals events. The van optimizer chooses among feasible candidate plans. The transfer rule decides whether an eligible request moves to Lyft. [optional] The simulator holds the clock, the vans and every request's state; it reveals a request only when it arrives, and applies a hidden draw only when its event happens. The decision step, on the right, sees only what has happened: the van dispatcher generates candidate routes, an optimizer—a mixed-integer linear program—picks one per van, and the transfer rule reads those plans. Then the vans execute, and every event is recorded. Future requests, each rider's patience and presence, and the real Lyft delay stay hidden from the dispatcher.

TRANSITION

These parts work together in one repeating cycle.

Slide 15 · The same decision cycle repeats as the evening unfolds

about 58 s · open slide

SAY (exact narration)

The dispatcher sees requests that have arrived. It does not see future requests. Every two minutes, and at each new request, waiting-time cancellation or no-show, or van coming on duty, the same cycle runs. Reveal and update. Generate feasible routes for each van. Select one route per van. Evaluate the transfer rule for each request still waiting. Apply accepted transfers: those requests leave van planning, and the Lyft is ordered two minutes later. If anything was transferred, re-plan once. Then vans execute and commit to their next stop. The rule runs once per decision moment, so there is no cascade of transfers in the same second.

TRANSITION

How does the optimizer pick a route?

Slide 16 · The optimizer compares the consequences of candidate plans

about 76 s · open slide

SAY (exact narration)

Waiting, riding, driving and unresolved requests enter the model in different ways. Waiting is weighted by party size and is convex: each extra minute costs more the longer someone has waited. Riding, at half a point per passenger-minute, and driving, at a tenth per kilometre, always count less than waiting. A request given to no van in this round is postponed; postponing has a price, plus a large constant, so the plan first serves as many requests as its candidates allow. Seats and pickup-before-drop-off are hard constraints; ride time has a softer guardrail. In this example, picking up the rider who has waited eight minutes first costs 66.5 points; the other order costs 81. [optional] That rider would pass fifteen minutes. 'Optimal' means best among the candidate plans generated at that decision moment—not the best possible whole evening.

TRANSITION

The van optimizer stays fixed. What we change is the transfer rule.

Slide 17 · We changed the transfer decision, not the van dispatcher

about 69 s · open slide

SAY (exact narration)

We tested six rules, one at a time. Vans only never transfers. Wait ten minutes and Wait fifteen minutes move a request once it has waited that long. Unassigned for ten minutes moves a request left out of every van plan for ten minutes in a row. Predicted total wait exceeds fifteen minutes uses the van plan: predicted total wait equals predicted pickup clock time minus request clock time. And Lyft arrives sooner fires if the predicted van pickup is later than now plus the two-minute ordering lag, an eight-minute typical Lyft delay and a two-minute margin. We call these last two the predictive rules; the first ones use only time. Only requests still waiting, not committed to a van, are eligible. The thresholds were specified before any run.

TRANSITION

Here is that last rule, worked through.

Slide 18 · What does ‘Lyft arrives sooner’ actually compare?

about 64 s · open slide

SAY (exact narration)

The rule compares predicted van pickup with expected Lyft pickup, including the ordering delay. On a small designed evening, a request arrives at about 6:16. At 6:17 the van plan predicts pickup at 6:30 and eight seconds. A Lyft ordered then is expected at 6:27: two minutes to place the order plus a typical eight-minute delay. That is three minutes eight seconds sooner, more than the two-minute margin, so the request moves. The Lyft is ordered at 6:19 and reaches the curb at 6:27, ten and a half minutes after the request. The van pickup time is a prediction. We do not observe what that van would have done after choosing a different action.

TRANSITION

Testing at this level of detail also exposed a weakness.

Slide 19 · A cheap postponement price could keep a rider waiting

about 60 s · open slide

SAY (exact narration)

The first formulation could price postponement more cheaply than a physically possible pickup. At 6:15 on the designed evening, a party of five had waited fifteen minutes. Postponing them was priced at 262.5 points, but the earliest pickup any van could actually make was worth about 436. We compared three versions over that whole evening and adopted the one that removes the flaw without adding variables: price postponement at the earliest time any van could physically arrive. The adopted approximation reduced the worst wait in this designed case. It did not reduce average waiting. And it is still optimistic, because it ignores the nearest van’s other work.

TRANSITION

With the model settled, how do we compare rules fairly?

Slide 20 · Replay the same night; change one rule

about 44 s · open slide

SAY (exact narration)

Each rule faces the same requests and hidden rider inputs, so differences reflect the rule within this specified model. We replay each constructed night six times, with the same requests, hidden traits, Lyft delays and vans, changing only the rule. Thirty nights replayed six ways are thirty paired scenarios—not 180 independent nights. Pairing removes night-to-night differences from each comparison; it does not make the nights representative of April. The settings are provisional, including the placeholder patience.

TRANSITION

Before trusting any result, we checked the implementation.

Slide 21 · We checked implementation before interpreting outcomes

about 54 s · open slide

SAY (exact narration)

Verification asks whether the code implements the specified model. Validation asks whether the model represents the real service. On verification: three small designed evenings reproduce hand-computed event times to the second. A separate brute-force program checked about 290,000 plan combinations on a designed evening and always agreed with the optimizer. 253 automated tests pass, and the reruns we sampled were byte-identical. We should be clear: AI wrote the simulator code and the checks, under our direction. Separate implementations are useful checks, but they share our assumptions and do not replace our own understanding. Validation against real riders is still ahead.

HANDOFF: Youzhi Zhang → Vignesh Rajesh Babu

We then replayed the same evenings under different transfer rules. Vignesh will explain what changed, what it means and what remains uncertain.

6 Manual verification activity: One decision moment on a small designed evening

> **WHAT HUMAN VERIFICATION MEANS**
>
> A program can generate and check an answer. Human verification means a teammate independently works through a defined example, records their calculation and compares it with the implementation.

This is a guided preparation activity, not a quiz or an exam. Work through the procedure on your own first and write down your result. Only then turn to the reference answer, which is on a separate page. If your result differs, record the difference rather than adjusting your working to match: a disagreement is useful information. The reference answer was produced with AI assistance from the project files; it does not count as a teammate's verification.

Related slides: Slides 15–16. **Goal:** Check by hand one complete decision of the van optimizer on the small designed evening **L1**: request **R004** at decision moment τ **= 1,853 s (18:30:53)**. You will list the feasible choices, check order and capacity, compute each objective term, and identify the selected plan.

Source inputs

The situation at 18:30:53 (τ = 1,853 s after 18:00). On L1, requests are designed to arrive only when both vans are idle, so the optimizer faces a single open request.

Item	Value
Van V1	idle at location `p05_b033`, free now, no riders aboard
Van V2	idle at location `p03_b016`, free now, no riders aboard
Open request R004	requested at 1,853 s; party of 1; pickup `p04_b076`; drop-off `p05_b015`
Seats	14 per van
Dwell	30 s at the pickup, counted in the ride

Travel table (road-network times rounded to whole seconds; distances in metres; travel multiplier 1.0 on L1):

Leg	Seconds	Metres
V1 location `p05_b033` $\rightarrow$ pickup `p04_b076`	278	2,041.3
V2 location `p03_b016` $\rightarrow$ pickup `p04_b076`	56	383.0
Pickup `p04_b076` $\rightarrow$ drop-off `p05_b015`	309	2,449.9

Scoring rules (slide 16). For a route: points = party $\times$ φ(wait in minutes) + 0.5 $\times$ party $\times$ ride minutes + 0.1 $\times$ km driven.

- φ: 1 point per minute for the first 5 minutes of waiting (then 2.5, 5, 10, 20 after 5, 10, 15, 20 minutes).

- Wait = vehicle's curb arrival − request time. Ride = 30-s dwell + pickup-to-drop-off travel.
- km driven = all legs of the route (to the pickup, then to the drop-off).
- Ride-time guardrail: ride must not exceed max(25 min, 2 × direct + 5 min). Direct time for R004 is 309 s.
- Leaving R004 unserved (postponed) costs at least the large constant M, which at this moment is 15,678,893 internal units = 13.07 points (internal units are points × 1,200,000).

Sources (for anyone with repository access):

- `data/scenarios/v2/L1/requests.csv` and `handcheck.csv` (row R004)
- `outputs/sim/L1_R0/epochs.jsonl` (the record with τ = 1853)
- `data/processed/tt_locations_v2.csv`

Procedure

1. List every feasible choice for this moment. (Hint: each van either takes R004 or stays idle, and R004 is served at most once. Include "postpone R004".)
2. For each choice that serves R004, check: pickup comes before drop-off; the van's load stays within 14 seats; the ride respects the guardrail.
3. For "V1 serves R004": compute the wait (minutes), the waiting term, the ride minutes, the riding term, the km driven and the driving term. Add them.
4. Repeat for "V2 serves R004".
5. Compare the totals with each other and with the postponement cost, and identify the plan the optimizer should choose.
6. Write the predicted pickup and drop-off clock times for the chosen plan.

Worksheet

YOUR WORKING

Step-by-step working (use the lines, or attach your own sheet)

My result

Record

Status	**Awaiting human completion**
Name	
Date completed	
My result	
Matches the reference? (yes / partly / no)	
Discrepancy, and what I did about it	
What remains unverified	

Reference answer

REFERENCE ANSWER (compare only after finishing your own working)

Feasible choices. (a) V1 serves R004, V2 idle; (b) V2 serves R004, V1 idle; (c) both idle, R004 postponed. Serving R004 with both vans is not allowed (served once).

Order and capacity. Each serving route is pickup then drop-off; load goes $0 \to 1 \to 0$, well within 14 seats. Ride = 30 + 309 = 339 s = 5.65 min, within the guardrail max(25, $2 \times 5.15 + 5 = 15.3$) = 25 min.

	(a) V1 serves	(b) V2 serves	(c) postpone
Wait	278 s = 4.6333 min	56 s = 0.9333 min	—
Waiting term, $1 \times \varphi$(wait)	4.6333	0.9333	—
Riding term, $0.5 \times 1 \times 5.65$	2.8250	2.8250	—
km driven	2.0413 + 2.4499 = 4.4912	0.3830 + 2.4499 = 2.8329	0
Driving term, $0.1 \times$ km	0.4491	0.2833	—
Total points	**7.9075**	**4.0416**	$\geq$ **13.07**

Selected plan: (b), V2 serves R004. Predicted pickup = 1,853 + 56 = 1,909 s (18:31:49); drop-off = 1,909 + 30 + 309 = 2,248 s (18:37:28).

Comparison with the implementation. The answer key (`handcheck.csv`, row R004) lists van V2, cost_V1 = 7.907453, cost_V2 = 4.041623, pickup 1,909 s, drop-off 2,248 s. The simulator's decision log at τ = 1,853 records the same two candidate routes (7.9075 and 4.0416 points, as 9,488,944 and 4,849,948 internal units), the V2 route chosen, an optimal solve and a unique optimum.

What this verifies, and what it does not. It verifies this one defined decision: the scoring arithmetic, the feasibility checks and the choice the optimizer made for R004 at 18:30:53. It does *not* verify the MILP in general, decisions with several open requests or riders aboard, the candidate-generation limit of three new pickups, the postponement price, or the rest of the evening.

Comparison checklist

- ☐ Did I list three choices, including postponement?
- ☐ Do my two serving totals match 7.9075 and 4.0416 within 0.001?
- ☐ Did I include the 30-s dwell in the ride minutes and both legs in the km?
- ☐ Did I select V2 and state why (lower total, far below the postponement cost)?
- ☐ Did I state that this checks one decision only, not the whole optimizer?

Record any disagreement in the table above. Do not change your own working to match the reference.

7 Rehearsal and readiness checklist

- ☐ Read my slides' preparation notes in the shared guide.
- ☐ Read my slides in the final deck (`02_Final_Presentation.pdf`) and know which backup slides I may need.
- ☐ Said my narration aloud with a stopwatch; my section took _____ (estimate 8:10).
- ☐ Practised my opening and handoff sentences with the neighbouring presenter.
- ☐ Can explain every concept listed above without the slides.
- ☐ Completed my verification activity and filled in the record (status changed from "Awaiting human completion" only by me).
- ☐ Took part in one timed full-team rehearsal (total time: _____).